

Normalization Confluence for Registry-Governed Stream Processing

Dayna Blackwell

dayna@blackwell-systems.com

Licensed under CC-BY-4.0.

5 October 2026

Version 2. See Appendix (Errata and corrections) for changes from version 1.

Abstract

Coordination-free convergence in distributed systems has been achieved through two structural regimes: *operation commutativity* (CRDTs), where operations commute by construction, and *invariant confluence* (I-confluence), where all orderings preserve application invariants. We identify a third regime—*normalization confluence*—in which operations may be non-commutative and may violate invariants, but convergence is restored through a compensation operator that normalizes states to validity.

We formalize this regime by introducing *registry-governed stream processing*, where an external registry of invariant predicates defines valid states and a compensation operator repairs violations. We construct a *governance rewrite system* whose configurations pair a state with a buffer of pending events, and whose rewrite rules include both event application (in any causality-respecting order) and compensation. Two structural conditions—*well-founded compensation* (WFC) and *compensation commutativity* (CC)—ensure termination and confluence respectively. Under both, Newman’s Lemma guarantees unique normal forms: any two processors consuming the same events agree on a valid state, regardless of the order in which they apply those events. A third condition—*uniformly bounded compensation* (UBC)—yields constant per-event overhead and $O(n \log n)$ total cost matching conventional stream processing. We show UBC is tight: without it, compensation depth grows without limit. We show CC is exact where it can be: with every buffered event free to fire and compensation that repairs to validity, every event set delivered from the initial state has a unique normal form if and only if CC holds on the reachable states; when causal dependencies constrain the future of a run, the exact condition is a joinability form of CC.

Compensation commutativity is strictly weaker than operation commutativity: operations need not commute, only their compensated results. The gap between these two conditions is precisely the design space that normalization confluence occupies.

To make CC practically verifiable, we develop a *verification calculus* that reduces the global CC obligation to local checks. An *invariant footprint* mapping identifies which invariants each event can affect, and a *decomposable repair* condition ensures per-invariant repairs commute. Under decomposable repair, CC2 is equivalent to a per-event strong absorption property, and CC1 for a pair reduces to commutation of the two events up to normalization plus a per-event footprint absorption check; at valid states, disjoint footprints and repair locality reduce CC1 to commutation up to normalization alone. A product composition theorem enables modular verification of multi-registry systems. The convergence theorems, their exact converses, the calculus, and the paper’s examples and counterexamples are mechanized in Coq/Rocq, axiom-free.

Keywords: Stream Processing, Normalization Confluence, Semantic Consistency, Rewrite Systems, Newman's Lemma, Compensation Commutativity, Distributed Systems

1 Introduction

Coordination-free convergence—the ability of distributed processors to agree on a result without explicit synchronization—is among the central goals of distributed systems design. The literature has identified two structural regimes under which it is achievable:

1. **Operation commutativity** (CRDTs [2]): Operations are designed to commute algebraically. Any application order produces the same state.
2. **Invariant confluence** (I-confluence [5]): Operations individually preserve application invariants under all orderings. No repair is needed because validity is never violated.

Both regimes achieve convergence by *preventing* inconsistency. CRDTs prevent it through algebraic design; I-confluence prevents it through invariant-preserving operation selection. Neither accommodates operations that are non-commutative *and* may violate invariants—yet such operations are pervasive in practice. An order processing pipeline, for example, must handle events (orders, payments, inventory adjustments, shipments) that do not commute and that individually violate business rules (no shipment without payment, no negative inventory).

We formalize the missing piece as a *registry*: any authoritative definition of validity external to the stream processor. Registries already exist in practice under various names—policy engines, constraint services, schema validators, compliance controllers—but their role in convergence has not been characterized formally. Our model abstracts these as a tuple of invariant predicates and a compensation operator.

This paper identifies a third regime: **normalization confluence**. Operations may be non-commutative. They may violate invariants. But a compensation operator normalizes states to validity after each operation, and the key structural condition is that these *compensated results* commute—even though the operations themselves do not. Under this condition, together with well-founded compensation, Newman’s Lemma guarantees that all causality-respecting processing orders reach the same valid state.

The three regimes compare as follows:

1. Operation commutativity (every state valid) $\implies$ compensation commutativity $\implies$ convergence.
2. Invariant confluence $\implies$ no compensation needed; in the sequential-application model of this paper, convergence then still requires the operations to commute on the reachable states.
3. Well-founded compensation and compensation commutativity $\implies$ convergence (this paper).

Normalization confluence occupies the design space between CRDT commutativity (which requires operations to commute) and I-confluence (which requires operations to preserve invariants). It permits operations that satisfy neither, provided their compensated results agree.

1.1 Convergence via Normalization

Convergence is conventionally treated as a property of *operations*: operations must be designed to commute, or designed to preserve invariants. Our result reframes convergence as a property of the *normalization rewrite system*: the system that interleaves event application with invariant repair. The operations themselves need no special algebraic structure. What matters is that the rewrite system—encompassing both application and compensation—is terminating and confluent. This shifts the design burden from operation algebra to compensation design.

1.2 Contributions

1. **Normalization confluence regime** (Section 3): A governance rewrite system on configurations (σ, B) pairing a state with a pending-event buffer, with rules for both event application and compensation. Nondeterminism arises from the choice of which enabled event to apply next.
2. **Convergence theorem** (Section 5): Proof that processors consuming the same events converge to the same valid state, regardless of application order, under well-founded compensation (WFC) and compensation commutativity (CC).
3. **Tightness and exactness** (Section 6): without UBC, compensation depth is unbounded; a failure of CC can make different orders reach different valid states; and CC is the exact condition (an if-and-only-if) when every buffered event may fire and compensation repairs to validity, with a joinability form of CC exact under any enabledness.
4. **Complexity analysis** (Section 7): Proof that registry governance adds $O(\log n)$ overhead per event under UBC, preserving practical scalability.
5. **Three-regime characterization** (Section 8): Together with CRDTs and invariant confluence, a partition of the coordination-free design space into operation commutativity, invariant confluence, and normalization confluence.
6. **Verification calculus** (Section 9): invariant footprints and decomposable repair reduce the global CC obligation to per-event checks (strong absorption, footprint absorption) and per-pair commutation up to normalization, with each hypothesis shown to be needed. Product composition theorems enable modular verification of multi-registry systems.
7. **Machine-checked proof** (Remark 5.4): A Coq/Rocq mechanization of the Convergence Theorem (Newman’s Lemma and the three critical-pair cases), of the stream-level theorem, of the exact converses, of the calculus, and of every worked example and counterexample, all axiom-free (`Print Assumptions` reports “Closed under the global context”), with concrete registries witnessing that the hypotheses are jointly satisfiable, so no theorem is vacuous.

2 Related Work

Conflict-Free Replicated Data Types. CRDTs [2] guarantee convergence through algebraic structure: replicas that apply the same set of commutative, associative, idempotent operations converge to the same state. Our setting is strictly more general in one dimension: we do not require operations to commute, only that their *compensated results* commute. This admits raw operations that leave the valid states and are then repaired, a convergence mechanism CRDTs do not use: a CRDT converges by the algebra of its operations, with no repair. With compensation removed the two coincide, and the extension is strict on the same transition representation, not on observable behavior (Section 8.1). Conversely, CRDTs require no finiteness or boundedness assumptions. Neither approach dominates: CRDTs are preferable when operations can be designed to commute, while registry governance is preferable when business rules constrain valid states in ways that preclude commutativity.

CALM and Coordination Avoidance. The CALM theorem [3, 4] establishes that monotone programs can achieve consistency without coordination. Bailis et al. [5] extend this to identify *invariant confluence*: a set of operations is invariant-confluent if all possible orderings preserve application invariants. When invariant confluence holds, no compensation is needed. Our framework addresses the complement: operations that are *not* invariant-confluent. We prove that compensation restores convergence under well-founded compensation and compensation commutativity. Section 8.1 compares the two regimes precisely.

Invariant-Repair Systems. Indigo [15] and IPA [16] also restore application invariants over weakly consistent replicas. They do so by a different route: static analysis of invariant-violating operation pairs, to which they attach application-specific conflict-resolution rules or reservation (escrow) protocols so that every reachable state stays invariant-preserving, in the invariant-confluence tradition. We do not preselect operations to be invariant-preserving. We admit operations that genuinely violate invariants and prove, via Newman’s Lemma under compensation commutativity, that repair reconciles all orderings to the same valid normal form. The distinction is one of method: reservation and invariant confluence versus confluence of a compensating rewrite system.

Abstract Rewrite Systems. Newman’s Lemma [1] states that a terminating abstract rewrite system is confluent if and only if it is locally confluent. The lemma has been applied extensively in term rewriting [6] and lambda calculus. Our contribution is its application to a rewrite system with *genuine nondeterminism*: the choice of which event to apply next from a buffer of enabled events. Local confluence is nontrivial because different application orders produce different intermediate states; our compensation commutativity axiom is the condition that closes the resulting critical pairs.

Rewriting Logic and Coherence. Meseguer’s rewriting logic [17] models concurrent systems as rewrite theories, and coherence checking (the Church-Rosser and coherence properties of rewrite theories modulo equations [18], supported by tools such as Maude) is the closest methodological ancestor of our build-time convergence check. We instantiate this lineage for one specific rewrite system: events interleaved with invariant compensation, where compensation commutativity plays the role of the coherence condition that closes critical pairs.

Self-Stabilization. Self-stabilizing systems [19] converge to a legitimate state from an arbitrary starting configuration. Normalization confluence shares the converge-to-valid premise but strengthens the conclusion to agreement: not merely that each replica reaches some valid state, but that all replicas reach the *same* valid state under reordering.

Eventual Consistency. Classical eventual consistency [7] guarantees that replicas converge given sufficient time without new updates. It does not address whether converged states satisfy application-level invariants—only that replicas agree on the same value. Our work guarantees convergence to *valid* states under continuous event arrival.

Stream Processing Theory. Formal models of stream processing include synchronous dataflow [9], Kahn process networks [10], and the windowed computation models underlying modern engines [11]. These focus on operational semantics—what a stream processor computes—rather than semantic validity of computed states. Our registry layer sits above any of these operational models.

Saga Pattern and Compensating Transactions. The saga pattern [8] decomposes long-lived transactions into sequences of local transactions with compensating actions for rollback. Our compensation operator generalizes this: rather than undoing individual operations, it repairs arbitrary invariant violations in the current state. The convergence guarantee we prove—that different interleaving orders of application and compensation reach the same valid state—has no analogue in the saga literature.

Mechanized Convergence Proofs. Gomes et al. [20] give a machine-checked proof of strong eventual consistency for CRDTs in Isabelle/HOL, network model included. Their result covers the commutative case; the mechanization accompanying this work (Remark 5.4) covers convergence under non-commutative, invariant-violating operations reconciled by compensation. Both rest on an assumption-free, machine-checked convergence theorem rather than a paper proof.

3 Registry-Governed Stream Model

3.1 Registries and Semantic Projection

Definition 3.1 (Registry). A *registry* is a tuple $R = (\Sigma_R, I_R, V_R, \rho_R)$ where:

- Σ_R is a set of *semantic states*,
- $I_R = \{\psi_1, \dots, \psi_k\}$ is a finite set of *invariant predicates* $\psi_i : \Sigma_R \rightarrow \{\top, \perp\}$,
- $V_R : \Sigma_R \rightarrow \{\top, \perp\}$ is the *validation function* defined by $V_R(\sigma) = \bigwedge_{i=1}^k \psi_i(\sigma)$,
- $\rho_R : \Sigma_R \rightarrow \Sigma_R$ is a *compensation operator* satisfying: if $V_R(\sigma) = \perp$, then $\Phi(\rho_R(\sigma)) < \Phi(\sigma)$ for a well-founded measure Φ (defined below); if $V_R(\sigma) = \top$, then $\rho_R(\sigma) = \sigma$.

Definition 3.2 (Semantic Projection). A registry is defined over a semantic state space Σ_R together with a *projection* $\pi : \Sigma_{\text{raw}} \rightarrow \Sigma_R$ from unbounded raw states to semantic states. All invariants, validation, and compensation are defined on Σ_R . Two raw states s_1, s_2 are *registry-indistinguishable* if $\pi(s_1) = \pi(s_2)$. The application function $\text{apply} : \mathcal{E} \times \Sigma_R \rightarrow \Sigma_R$ is understood as “apply then project”: given a raw-level application $\text{apply}_{\text{raw}} : \mathcal{E} \times \Sigma_{\text{raw}} \rightarrow \Sigma_{\text{raw}}$, the semantic application is $\text{apply}(e, \sigma) = \pi(\text{apply}_{\text{raw}}(e, s))$ for any s with $\pi(s) = \sigma$. Well-definedness requires that the projected result is independent of the representative s .

Remark 3.3 (Scope of Finiteness). Finiteness is required of the *semantic* state space Σ_R , not the raw data. An order-processing registry tracking status across states *pending*, *paid*, *shipped*, *delivered*, *cancelled* has $|\Sigma_R| = 5$ regardless of how many bytes the underlying order record occupies. More generally, any domain whose logic is captured by k boolean predicates has $|\Sigma_R| \leq 2^k$: the semantic states are the equivalence classes under the invariant vector $(\psi_1(\sigma), \dots, \psi_k(\sigma))$. Business domains are overwhelmingly of this form.

Definition 3.4 (Registry Equivalence). Two states $\sigma_1, \sigma_2 \in \Sigma_R$ are *registry equivalent*, written $\sigma_1 \approx_R \sigma_2$, if they satisfy the same invariants: $\forall \psi \in I_R : \psi(\sigma_1) = \psi(\sigma_2)$.

Lemma 3.5. *Registry equivalence $\approx_R$ is an equivalence relation on Σ_R .*

Proof. Each condition $\psi(\sigma_1) = \psi(\sigma_2)$ is reflexive, symmetric, and transitive. A conjunction of equivalence relations is an equivalence relation. $\square$

3.2 Well-Founded and Uniformly Bounded Compensation

Termination and complexity require two properties of the compensation operator, which we state separately because they play different roles.

Axiom 3.6 (Well-Founded Compensation (WFC)). There exists a *compensation measure* $\Phi : \Sigma_R \rightarrow \mathbb{N}$ such that every compensation step strictly decreases Φ : if $V_R(\sigma) = \perp$, then $\Phi(\rho_R(\sigma)) < \Phi(\sigma)$.

WFC guarantees that compensation terminates from any state: iterated application of ρ_R reaches a valid state in finitely many steps.

Axiom 3.7 (Uniformly Bounded Compensation (UBC)). The compensation measure Φ from Axiom 3.6 is uniformly bounded: there exists $M \in \mathbb{N}$ such that $\Phi(\sigma) \leq M$ for all $\sigma \in \Sigma_R$.

UBC strengthens WFC by bounding compensation depth *uniformly* across all states. WFC alone suffices for termination and for defining iterated compensation ρ_R^* . UBC is additionally needed for the uniform step bound in the termination lemma and the constant per-event compensation cost in the complexity analysis.

Lemma 3.8 (Finite State Spaces Imply UBC). *If $|\Sigma_R| < \infty$ and WFC holds, then UBC holds with $M = \max_{\sigma \in \Sigma_R} \Phi(\sigma)$.*

Proof. The maximum exists because Σ_R is finite. □

Remark 3.9. WFC is the load-bearing condition for termination; UBC is the load-bearing condition for uniform step bounds and per-event complexity. Finiteness of Σ_R is a convenient sufficient condition for both, but UBC does not require full finiteness of Σ_R ; it requires only bounded compensation height. The counterexample in Section 6 satisfies WFC (compensation terminates for every fixed event set) but violates UBC (compensation depth grows without bound as event sets grow).

3.3 Events and Streams

Definition 3.10 (Event Stream). An *event stream* $\mathcal{S} = \langle e_1, e_2, \dots \rangle$ is a (possibly infinite) sequence of events drawn from event space $\mathcal{E}$. Each event e_i carries a unique identifier $\text{id}(e_i)$, a timestamp $\tau(e_i)$, and a set of causal dependencies $\text{deps}(e_i) \subseteq \{e_1, \dots, e_{i-1}\}$. An *application function* $\text{apply} : \mathcal{E} \times \Sigma_R \rightarrow \Sigma_R$ maps each event and current state to a new state.

Definition 3.11 (Causality-Respecting Order). An ordering π of a finite event set E *respects causality* if whenever $e_i \in \text{deps}(e_j)$, event e_i precedes e_j under π . Multiple causality-respecting orders may exist for the same event set.

3.4 Governance Rewrite System

We now define the rewrite system that is the core technical device of this paper. Unlike the approach of fixing a canonical event order—which would make agreement trivial by determinism—our rewrite system admits genuine nondeterminism in the choice of which event to apply next.

Assumption 3.12 (Causal Closure). The received event set E is causally closed: for every $e \in E$, $\text{deps}(e) \subseteq E$.

Under causal closure, every dependency of every event in E is either still pending in B or has already been applied (removed from B). This makes the enabled predicate below well-defined.

Definition 3.13 (Configuration). A *configuration* is a pair (σ, B) where $\sigma \in \Sigma_R$ is a semantic state and $B \subseteq E$ is a set of events that have been received but not yet applied. An event $e \in B$ is *enabled* if $\text{deps}(e) \cap B = \emptyset$ (all causal dependencies of e have already been applied).

Definition 3.14 (Governance Rewrite System). Fix a finite set of received events E and initial state σ_0 . The *governance rewrite system* $\mathcal{G} = (\mathcal{C}, \rightarrow)$ operates on configurations (σ, B) with $\sigma \in \Sigma_R$ and $B \subseteq E$. The rewrite relation $\rightarrow$ is the union of:

1. **Apply step:** $(\sigma, B) \rightarrow (\text{apply}(e, \sigma), B \setminus \{e\})$ whenever $e \in B$ is enabled.

2. Compensation step: $(\sigma, B) \rightarrow (\rho_R(\sigma), B)$ whenever $V_R(\sigma) = \perp$.

A configuration is in *normal form* if $B = \emptyset$ and $V_R(\sigma) = \top$.

The nondeterminism is genuine: when multiple events are enabled and the state is invalid, a configuration may step via different apply rules (choosing different events) or via the compensation rule. Confluence of this system means that the choice does not affect the final result.

3.5 Compensation Commutativity

Local confluence of $\mathcal{G}$ is nontrivial. Two critical pair types arise: (1) two apply steps choosing different events, and (2) an apply step versus a compensation step. The following axiom ensures both types close.

Definition 3.15 (Iterated Compensation). Under WFC (Axiom 3.6), define $\rho_R^*(\sigma)$ to be the unique state obtained by iterating ρ_R from σ until reaching validity: $\rho_R^*(\sigma) = \rho_R^m(\sigma)$ for the least m such that $V_R(\rho_R^m(\sigma)) = \top$. Existence and finiteness of m follow from WFC (indeed $m \leq \Phi(\sigma)$); uniqueness follows from determinism of ρ_R . Because V_R is Boolean-valued, ρ_R^* is computable, and it does not depend on which measure witnesses WFC.

Axiom 3.16 (Compensation Commutativity (CC)). The registry R satisfies:

(CC1) Order independence. For any $\sigma \in \Sigma_R$ and events e_1, e_2 that are causally independent and simultaneously enabled in some configuration (σ, B) :

$$\rho_R^*(\text{apply}(e_2, \rho_R^*(\text{apply}(e_1, \sigma)))) = \rho_R^*(\text{apply}(e_1, \rho_R^*(\text{apply}(e_2, \sigma)))).$$

(CC2) Compensation absorption. For any $\sigma \in \Sigma_R$ with $V_R(\sigma) = \perp$ and any enabled event e :

$$\rho_R^*(\text{apply}(e, \sigma)) = \rho_R^*(\text{apply}(e, \rho_R(\sigma))).$$

Remark 3.17 (CRDT Commutativity). CRDT commutativity requires $\text{apply}(e_1, \text{apply}(e_2, \sigma)) = \text{apply}(e_2, \text{apply}(e_1, \sigma))$ for all σ . This implies CC1 (take $\rho_R^* = \text{id}$ when all states are valid) and trivially satisfies CC2 (when all states are valid, compensation never fires). Compensation commutativity is strictly weaker: it permits operations that individually violate invariants, requiring only that the *repaired* results commute. The gap between operation commutativity and compensation commutativity is precisely the space our result occupies.

Remark 3.18 (Compensation Absorption). Condition CC2 states that compensating an invalid state before applying an event does not change the final compensated result. Intuitively, the repair that compensation performs “would have been performed anyway” during the post-application compensation phase. CC2 holds whenever compensation repairs invariant violations in a way that is determined by the violation pattern rather than the path taken to reach the violation. By induction on the number of compensation steps, CC2 generalizes: $\rho_R^*(\text{apply}(e, \sigma)) = \rho_R^*(\text{apply}(e, \rho_R^*(\sigma)))$. In rewrite-theoretic terms, CC2 ensures that the normalizer is stable under interleaving with event application: normalizing “early” (before an event) versus “late” (after) does not change the final normal form.

3.6 Stream Processors

Definition 3.19 (Stream Processor). A *stream processor* P for stream $\mathcal{S}$ under registry R is a process that, at each time t , has received some subset of events $E_P(t) \subseteq \mathcal{S}$ and maintains state $\Psi_P(t) \in \Sigma_R$. We require:

1. **Eventual delivery:** For every event $e_i \in \mathcal{S}$, there exists a time t_i such that $e_i \in E_P(t)$ for all $t \geq t_i$.
2. **Causality-respecting application:** The processor applies events in some order that respects causality (Definition 3.11), buffering events whose causal dependencies have not yet been received. The processor is free to choose any causality-respecting order.
3. **Incremental compensation:** After applying each event, the processor applies ρ_R until validity is restored.

The processor’s computation on event set $E_P(t)$ is a reduction sequence in the governance rewrite system $\mathcal{G}$ starting from $(\sigma_0, E_P(t))$. The processor has *finished processing* $E_P(t)$ at time t if its configuration at t is a normal form of that reduction. It is *fair* if, while no new events arrive, it keeps reducing, and makes progress whenever it has not finished.

A received set that is not yet causally closed (an event has arrived before one of its dependencies) is never finished in this sense: the waiting event stays in the buffer, so the configuration is not a normal form. Different processors may apply the same events in different causality-respecting orders. The convergence theorem guarantees that, once they have finished processing the same events, they hold the same valid state.

4 Worked Example: Order Fulfillment

We instantiate the framework on an order fulfillment pipeline, verify the structural conditions, and trace two processor orderings to the same valid state.

4.1 Registry Definition

An order has a *stage* in $\{\text{pending}, \text{approved}, \text{held}\}$ and a *balance* in $\{0, 1, 2, 3\}$. The semantic state space is $\Sigma_R = \{\text{pending}, \text{approved}, \text{held}\} \times \{0, 1, 2, 3\}$, with $|\Sigma_R| = 12$ and initial state $\sigma_0 = (\text{pending}, 0)$.

A single invariant governs fulfillment:

$$\psi(s, b) = \top \iff (s = \text{approved} \implies b \geq 2).$$

Approval requires a balance of at least 2. The only invalid states are $(\text{approved}, 0)$ and $(\text{approved}, 1)$.

Compensation. When an order is approved with insufficient balance, it is placed on hold:

$$\rho(\text{approved}, b) = (\text{held}, b) \quad \text{for } b < 2.$$

All other states are valid, and ρ acts as the identity. The *held* stage preserves the intent to approve: the system records that approval was requested but cannot yet proceed. This is a deliberate design choice—a naive alternative that simply cancels the approval (reverting to *pending*) would violate CC, as we discuss below.

Measures. $\Phi(s, b) = 1$ if $s = \text{approved}$ and $b < 2$, otherwise 0. WFC holds (ρ strictly decreases Φ on invalid states), and UBC holds with $M = 1$.

4.2 Events

Two causally independent events:

- e_{credit} : Adds 1 to the balance (capped at 3). If the order is on hold and the new balance reaches 2, the order is automatically approved:

$$\text{apply}(e_{\text{credit}}, (s, b)) = \begin{cases} (\text{approved}, \min(b+1, 3)) & \text{if } s = \text{held} \text{ and } \min(b+1, 3) \geq 2, \\ (s, \min(b+1, 3)) & \text{otherwise.} \end{cases}$$

- e_{approve} : Sets the stage to *approved*: $\text{apply}(e_{\text{approve}}, (s, b)) = (\text{approved}, b)$.

4.3 Verifying CC

CC1 (Order Independence). We verify CC1 for $(e_{\text{approve}}, e_{\text{credit}})$ at two representative starting states.

From $(\text{pending}, 0)$:

Path 1 (e_{approve} first):

$$(\text{pending}, 0) \xrightarrow{e_{\text{approve}}} (\text{approved}, 0) \xrightarrow{\rho^*} (\text{held}, 0) \xrightarrow{e_{\text{credit}}} (\text{held}, 1). \quad \text{Result: } (\text{held}, 1).$$

Path 2 (e_{credit} first):

$$(\text{pending}, 0) \xrightarrow{e_{\text{credit}}} (\text{pending}, 1) \xrightarrow{e_{\text{approve}}} (\text{approved}, 1) \xrightarrow{\rho^*} (\text{held}, 1). \quad \text{Result: } (\text{held}, 1). \quad \checkmark$$

In Path 1, approval fires first, violates the invariant (balance $0 < 2$), and compensation places the order on hold. The subsequent credit increments the balance but does not yet reach the threshold. In Path 2, credit arrives first (no violation), then approval violates the invariant and compensation again produces the held state. Both paths reach $(\text{held}, 1)$.

From $(\text{pending}, 1)$:

$$\text{Path 1: } (\text{pending}, 1) \xrightarrow{e_{\text{approve}}} (\text{approved}, 1) \xrightarrow{\rho^*} (\text{held}, 1) \xrightarrow{e_{\text{credit}}} (\text{approved}, 2). \quad \text{Result: } (\text{approved}, 2).$$

$$\text{Path 2: } (\text{pending}, 1) \xrightarrow{e_{\text{credit}}} (\text{pending}, 2) \xrightarrow{e_{\text{approve}}} (\text{approved}, 2). \quad \text{Result: } (\text{approved}, 2). \quad \checkmark$$

In Path 1, approval again violates the invariant; compensation holds; the subsequent credit reaches balance 2, triggering auto-approval. In Path 2, the credit arrives first, and approval at balance 2 is valid. Both reach $(\text{approved}, 2)$: the order is approved with sufficient funds.

Since $|\Sigma_R| = 12$, CC1 can be verified exhaustively; it holds for every pair of events at all 12 states, invalid ones included. All cases follow the same pattern: compensation preserves the approval intent in the held state, and subsequent credits eventually satisfy the balance threshold.

CC2 (Compensation Absorption). From the invalid state $(\text{approved}, 0)$ with event e_{credit} :

$$\text{LHS: } \rho^*(\text{apply}(e_{\text{credit}}, (\text{approved}, 0))) = \rho^*((\text{approved}, 1)) = (\text{held}, 1).$$

$$\text{RHS: } \rho^*(\text{apply}(e_{\text{credit}}, \rho(\text{approved}, 0))) = \rho^*(\text{apply}(e_{\text{credit}}, (\text{held}, 0))) = \rho^*((\text{held}, 1)) = (\text{held}, 1). \quad \checkmark$$

Compensating before the credit (putting the order on hold first) produces the same result as applying the credit to the invalid state and then compensating. CC2 holds because the held state and the approved-but-invalid state respond identically to subsequent credits after compensation.

4.4 Two Processors, Three Events

Consider the stream $\mathcal{S} = \langle e_{\text{approve}}, e_{\text{credit}}, e_{\text{credit}} \rangle$ (approval followed by two credits). Two processors P_1, P_2 receive the events in different orders:

Processor P_1 (approve, credit, credit):

$$(pending, 0) \xrightarrow{e_{\text{approve}}} (approved, 0) \xrightarrow{\rho^*} (held, 0) \xrightarrow{e_{\text{credit}}} (held, 1) \xrightarrow{e_{\text{credit}}} (approved, 2).$$

Processor P_2 (credit, credit, approve):

$$(pending, 0) \xrightarrow{e_{\text{credit}}} (pending, 1) \xrightarrow{e_{\text{credit}}} (pending, 2) \xrightarrow{e_{\text{approve}}} (approved, 2).$$

Both processors arrive at $(approved, 2)$. P_1 encountered an invariant violation (approval without sufficient balance) and passed through the held state; P_2 never violated the invariant. Compensation commutativity guarantees agreement regardless of the path.

4.5 Why Compensation Design Matters

The held stage is essential. A naive compensation that simply reverts approval to *pending*, that is, $\rho(approved, b) = (pending, b)$ for $b < 2$, satisfies WFC and UBC but violates CC1. The witness is the state $(pending, 1)$: approval then credit gives $\rho(approved, 1) = (pending, 1)$ and then $(pending, 2)$ (intent to approve lost), while credit then approval gives $(pending, 2)$ and then $(approved, 2)$, which is valid. So

$$\begin{aligned} \rho^*(\text{apply}(e_{\text{credit}}, \rho^*(\text{apply}(e_{\text{approve}}, (pending, 1))))) &= (pending, 2), \\ \rho^*(\text{apply}(e_{\text{approve}}, \rho^*(\text{apply}(e_{\text{credit}}, (pending, 1))))) &= (approved, 2) \neq (pending, 2). \end{aligned}$$

On the stream $\langle e_{\text{approve}}, e_{\text{credit}}, e_{\text{credit}} \rangle$ below, P_1 would end in $(pending, 2)$ and P_2 in $(approved, 2)$. (At $(pending, 0)$ the two orders happen to agree: both give $(pending, 1)$.) The naive revert also violates CC2: at the invalid state $(approved, 1)$,

$$\begin{aligned} \rho^*(\text{apply}(e_{\text{credit}}, (approved, 1))) &= (approved, 2), \\ \rho^*(\text{apply}(e_{\text{credit}}, \rho(approved, 1))) &= (pending, 2). \end{aligned}$$

The held stage preserves the approval intent so that subsequent credits can trigger auto-approval, ensuring that different orderings converge.

This illustrates the practical content of the CC conditions: they constrain *compensation design*, not operation design. What the held stage provides is *absorption*: compensating before a credit or after it gives the same normalized result (CC2), and the two events commute once results are normalized. Section 9 shows that these per-event and per-pair facts give CC1 at every state, and that under absorption CC1 is equivalent to commutation up to normalization (Theorem 9.10).

5 Convergence Theorem

5.1 Termination

Lemma 5.1 (Termination). *Under WFC (Axiom 3.6), the governance rewrite system $\mathcal{G}$ is terminating: every reduction sequence from (σ_0, E) is finite. Under UBC (Axiom 3.7), the total number of steps is bounded by $|E| + (|E| + 1) \cdot M$.*

Proof. Define the measure $\mu(\sigma, B) = (|B|, \Phi(\sigma))$ taking values in $\mathbb{N} \times \mathbb{N}$ under the lexicographic order, which is well-founded.

An apply step sends (σ, B) to $(\text{apply}(e, \sigma), B \setminus \{e\})$. The first component decreases: $|B \setminus \{e\}| = |B| - 1$. Therefore μ strictly decreases regardless of the second component.

A compensation step sends (σ, B) to $(\rho_R(\sigma), B)$. The first component is unchanged; the second strictly decreases by WFC. Therefore μ strictly decreases.

Since μ maps into a well-ordered set and strictly decreases at every step, every reduction sequence is finite. Under UBC, $\Phi(\sigma) \leq M$ for all σ , so at most M compensation steps can occur between apply steps (including before the first apply), giving the uniform bound $|E| + (|E| + 1) \cdot M$. $\square$

The bound is tight: a registry with $M = 1$ and a single event has a reduction of $3 = |E| + (|E| + 1)M$ steps (compensate, apply, compensate).

5.2 Local Confluence

Lemma 5.2 (Local Confluence). *Under CC (Axiom 3.16), the governance rewrite system $\mathcal{G}$ is locally confluent: if $(\sigma, B) \rightarrow X_1$ and $(\sigma, B) \rightarrow X_2$, then there exists a configuration Z with $X_1 \rightarrow^* Z$ and $X_2 \rightarrow^* Z$.*

Proof. Three cases arise from the two rewrite rules.

Case 1: Two apply steps. $(\sigma, B) \rightarrow (\text{apply}(e_1, \sigma), B \setminus \{e_1\}) = X_1$ and $(\sigma, B) \rightarrow (\text{apply}(e_2, \sigma), B \setminus \{e_2\}) = X_2$, where $e_1 \neq e_2$ are both enabled.

Since e_1 and e_2 are both enabled, their dependencies are disjoint from B . By Assumption 3.12, $e_1, e_2 \in E$, and since $e_1 \in B$ we have $e_1 \notin \text{deps}(e_2)$ (otherwise $\text{deps}(e_2) \cap B \neq \emptyset$, contradicting enabledness of e_2), and symmetrically $e_2 \notin \text{deps}(e_1)$. Therefore e_1 and e_2 are causally independent.

From X_1 : compensate to validity, obtaining $(\rho_R^*(\text{apply}(e_1, \sigma)), B \setminus \{e_1\})$; then apply e_2 (still enabled since its dependencies remain outside $B \setminus \{e_1\}$) and compensate, reaching

$$Z_1 = (\rho_R^*(\text{apply}(e_2, \rho_R^*(\text{apply}(e_1, \sigma)))), B \setminus \{e_1, e_2\}).$$

From X_2 : symmetrically, reach

$$Z_2 = (\rho_R^*(\text{apply}(e_1, \rho_R^*(\text{apply}(e_2, \sigma)))), B \setminus \{e_1, e_2\}).$$

By CC1, the state components of Z_1 and Z_2 are equal. Take $Z = Z_1 = Z_2$.

Case 2: Apply vs. compensation. $V_R(\sigma) = \perp$ and e is enabled in B . $(\sigma, B) \rightarrow (\text{apply}(e, \sigma), B \setminus \{e\}) = X_1$ and $(\sigma, B) \rightarrow (\rho_R(\sigma), B) = X_2$.

From X_1 : compensate to validity, reaching $(\rho_R^*(\text{apply}(e, \sigma)), B \setminus \{e\})$.

From $X_2 = (\rho_R(\sigma), B)$: apply e (still enabled, since compensation does not change B) to get $(\text{apply}(e, \rho_R(\sigma)), B \setminus \{e\})$; then compensate to validity, reaching $(\rho_R^*(\text{apply}(e, \rho_R(\sigma))), B \setminus \{e\})$.

By CC2, $\rho_R^*(\text{apply}(e, \sigma)) = \rho_R^*(\text{apply}(e, \rho_R(\sigma)))$. The buffer components are equal. Take $Z = (\rho_R^*(\text{apply}(e, \sigma)), B \setminus \{e\})$.

Case 3: Two compensation steps. Both rewrites apply ρ_R to the same state, producing the same successor $(\rho_R(\sigma), B)$. Take $Z = (\rho_R(\sigma), B)$. $\square$

5.3 Main Result

Corollary 5.3 (Unique Normal Forms). *Under WFC and CC, every configuration (σ_0, E) has a unique normal form.*

Proof. By Lemma 5.1, $\mathcal{G}$ is terminating. By Lemma 5.2, $\mathcal{G}$ is locally confluent. By Newman’s Lemma [1], $\mathcal{G}$ is globally confluent. A terminating, confluent rewrite system has unique normal forms. $\square$

Remark 5.4 (Machine-checked). This result and the rest of the paper’s theory are mechanized in Coq/Rocq (checked with Rocq 9.3; CI also runs Coq 8.18 and 8.20). Newman’s Lemma is proved from scratch (no external libraries) as *strongly normalizing + locally confluent $\implies$ confluent*, and the Governance Rewrite System is formalized with its two rules; the lexicographic measure $(|B|, \Phi(\sigma))$ discharges termination and the three critical-pair cases (closed by CC1, CC2, and determinism of ρ_R) discharge local confluence, yielding confluence and unique normal forms. The gate script `coq/verify.sh` checks 2012 named results at the time of this version, and `Print Assumptions` reports “Closed under the global context” (no axioms, no admitted lemmas) for every one. The results this paper relies on, by section:

- *Model and convergence.* ρ_R^* is constructed from WFC, as in Definition 3.15 (`base_def_rhostar`, `base_def_rhostar_least_unique`, `rho_star_canonical`; properties `rho_star_valid`, `rho_star_fix`, `rho_star_idem`), so Corollary 5.3 holds with WFC, CC1 on distinct, independent, co-enabled pairs, and CC2 as its only hypotheses, with the dependency-based enabledness of Definition 3.13 (`paper_causal_governance_unique_normal_forms`, `paper_governance_unique_normal_forms`; also `governance_unique_normal_forms`, `causal_governance_unique_normal_forms`). Termination and its bound: `base_lem_termination_bound`, attained by `lv_termination_bound_tight`; Lemma 3.8: `base_lem_finite_implies_ubc`; Remark 3.18: `base_rem_absorption`.
- *Stream level* (Definition 3.19, Theorem 5.5): `base_thm_convergence`, `stream_convergence`, `stream_agreement`, `base_rem_set_function`, `base_cor_quiescent`; counterexamples to the version-1 statements `base_thm_convergence_c_counterexample`, `base_thm_convergence_transient_counterexample`.
- *Domain independence* (Corollary 8.2): WFC over any well-founded order (`governance_wf_confluent`, `causal_governance_wf_confluent`, `base_cor_infinite`), with an instance on $\mathbb{Z}$ (`zw_confluent`, `zw_stream_agree`).
- *Tightness and exactness* (Section 6): `thm_necessity`, `ri_depth_exact`, `ri_no_ubc`, `ri_what_fails`, `ri_cc_any_extension`; `prop_cc_necessary`; the exact converses `cc_exact_from` (with ρ_R^* constructed: `wfc_cc_exact_from`) and `jc_exact`, and the qualifiers `masked_cc1`, `rho_star_qualifier`.
- *Worked example* (Section 4): `of_registry`, `of_cc1`, `of_cc2`, `of_paper_traces`, `of_processors`; the naive revert: `naive_cc1_fails`, `naive_cc2_fails`, `naive_stream_diverges`, `naive_paper_witness_fails`.
- *Complexity* (Theorem 7.1, model-level step bound): `base_thm_complexity_per_event`, `base_thm_complexity_comp_total`.
- *Calculus* (Section 9): `strong_absorption_iff_cc2`, `base_lem_repair_commute`, `base_lem_repair_idempotent`, the corrected footprint theorem and its parts (`calc_footprint_cc1_iff`, `calc_footprint_cc1_valid_iff`, `calc_footprint_cc1`, `calc_fp_absorb_iff_sa`, `calc_cc1_iff_commute_under_cc2`), the counterexamples to the version-1 statement (`base_thm_footprint_cc1_refuted`, `base_thm_footprint_cc1_raw_refuted`) and to dropping each hypothesis (`calc_valid_needs_commute`, `calc_valid_needs_disjoint`, `calc_valid_needs_repair_local`, `calc_all_needs_commute`, `calc_all_needs_absorb`, `calc_all_needs_repair_local`), the canonical-state pattern (`calc_canonical_pattern`,

base_pattern1_cc1_refuted), and product lifting (base_thm_product, base_thm_product_decomposable).

Each abstract theorem comes with a concrete registry discharging all of its hypotheses with genuine nondeterminism (for example `example_confluent`, `calc_clamp_instance`, `calc_order_cc1`), so no theorem is vacuous. The soundness of the reference implementation’s own certification strategy is mechanized as well: events whose state-variable footprints (read and write sets) are disjoint commute (`disjoint_events_commute`; this concerns state variables and is a different statement from the invariant footprints of Section 9), and a repair strictly decreasing a potential is strongly normalizing (`repair_terminates`). The development is furthermore paired with two runnable checkers extracted from it, each proven axiom-free: one certifies that a compiled machine’s emitted step tables commute and stay in range (`check_tables_converges`), and one recomputes convergence directly from the machine’s rules by evaluating their expression trees, trusting neither the implementation’s enumeration nor its normalization (`checkBuild_converges`). Because the checking logic is extracted from the proof rather than reimplemented by hand, the two serve as independent differential oracles for the reference implementation: a defect in its hand-written verification cannot make a non-convergent machine pass either checker. The proof scripts are available in the `coq/` directory of the accompanying artifact; `coq/PAPER-MAP.md` maps every numbered result of this paper to its Coq statement. Results mechanized after this paper was written (among them the distributed propagation model, root sets for lossy networks, exact state-based CRDTs, exact at-least-once delivery and exact stream agreement) are not written up here; `REGIME-AUDIT.md` in the artifact lists them with their Coq names and the gaps that remain open.

Theorem 5.5 (Stream Convergence). *Let R be a registry satisfying WFC (Axiom 3.6) and CC (Axiom 3.16), and let P_1, P_2 be stream processors (Definition 3.19) consuming the same event stream S . Then:*

- (a) **Validity:** *For each processor, after it completes the incremental compensation phase for any applied event, its state satisfies $V_R(\Psi_{P_j}(t)) = \top$.*
- (b) **Order independence:** *Once a processor has finished processing its received event set, its state depends only on that set, not on the order in which events were applied.*
- (c) **Agreement:** *Processors that have finished processing the same events are in the same state: if P_1 has finished processing $E_{P_1}(t_1)$, P_2 has finished processing $E_{P_2}(t_2)$, and $E_{P_1}(t_1) = E_{P_2}(t_2)$, then $\Psi_{P_1}(t_1) = \Psi_{P_2}(t_2)$. The times t_1 and t_2 may differ.*

Proof. We prove each part.

Part (a): Validity. By Lemma 5.1, compensation terminates. By Definition 3.19, the processor compensates after each event application until validity is restored. Therefore after completing the compensation phase for any applied event, $V_R(\Psi_{P_j}(t)) = \top$.

Part (b): Order independence. Each processor’s computation on event set $E = E_{P_j}(t)$ is a reduction sequence in the governance rewrite system from (σ_0, E) ; when the processor has finished, it has reached a normal form $(\sigma^*, \emptyset)$. Different causality-respecting orders correspond to different reduction sequences. By Corollary 5.3, all reduction sequences reach the same normal form. Therefore the final state σ^* depends only on E , not on the order.

Part (c): Agreement. If $E_{P_1}(t_1) = E_{P_2}(t_2) = E$ and both processors have finished, then both configurations are normal forms of reductions from (σ_0, E) . By Corollary 5.3, the normal form is unique, so $\Psi_{P_1}(t_1) = \Psi_{P_2}(t_2)$. $\square$

Remark 5.6 (Why “finished” is needed). Without the qualifier, part (c) fails: two processors can hold the same received set while one of them has not yet applied it. With a counter registry and the stream $\{e\}$, one processor that has applied e (state 1) and one that has received but not yet applied it (state 0, a zero-length reduction from $(0, \{e\})$) are both processors, both fair, and disagree. Eventual delivery alone also does not make received sets coincide on an infinite stream: if P_1 receives the n -th event at time n and P_2 at time $n + 1$, both can be finished at every time and still disagree at every time. What does hold is (c) as stated, at the same or at different times, and fixed-point agreement for finite streams (Corollary 5.8). Both counterexamples are mechanized (`base_thm_convergence_c_counterexample`, `base_thm_convergence_transient_counterexample`), as is the theorem (`base_thm_convergence_stream_agreement`).

Remark 5.7 (Semantic Projection). The theorem implies that the mapping $F(E) = \text{nf}(\sigma_0, E)$ —the unique normal form of the governance rewrite system starting from (σ_0, E) —is a well-defined function of the event set E , independent of causal linearization. This is the Church–Rosser property for our operational semantics.

Corollary 5.8 (Convergence Under Quiescent Streams). *If the stream $\mathcal{S}$ is finite and each processor is fair (it keeps processing while no new events arrive), then there exists a valid state σ^* such that both processors converge to σ^* : there exists T with $\Psi_{P_1}(t) = \Psi_{P_2}(t) = \sigma^*$ for all $t \geq T$.*

Proof. By eventual delivery, there exists T_N such that $E_{P_1}(t) = E_{P_2}(t) = \mathcal{S}$ for all $t \geq T_N$. From T_N on no new events arrive, so by fairness and Lemma 5.1 each processor finishes processing $\mathcal{S}$ at some time and stays at that normal form afterwards. By Theorem 5.5(c), from the later of the two finishing times T both processors are in the same valid state. $\square$

Remark 5.9 (Infinite Streams). For infinite streams, the received event sets grow monotonically toward $\mathcal{S}$ but may differ at every time, and eventual delivery does not make them coincide: a processor that is always one event behind another disagrees with it forever (Remark 5.6). The guarantee is Theorem 5.5(c): whenever two finished processors have received the same events, at the same or at different times, they hold the same state. The agreed-upon state may continue to change as new events arrive. Corollary 5.8 recovers fixed-point convergence when the stream is finite.

6 Necessity of the Structural Conditions

We show that UBC is necessary for uniform complexity, and that CC is the exact condition for agreement where an exact condition of this form exists.

6.1 Necessity of UBC

UBC is not merely convenient; it is necessary for a uniform bound on compensation depth, and thus for the constant per-event compensation overhead used in the complexity analysis (Section 7). Without it, compensation depth can grow without limit as event sets grow.

Theorem 6.1 (Unbounded Compensation Depth Without UBC). *For any $M \in \mathbb{N}$, there exists a registry R_∞ and a finite event set E such that some reduction sequence from (σ_0, E) in the governance rewrite system contains more than M compensation steps. Consequently, no uniform bound on compensation depth exists, and UBC fails.*

Proof. Define R_∞ with $\Sigma_{R_\infty} = \mathbb{Z}$, the single invariant $\psi(\sigma) = \top \iff \sigma = 0$, initial state $\sigma_0 = 0$, and compensation $\rho(\sigma) = \sigma + 1$ for $\sigma < 0$ and $\rho(\sigma) = \sigma - 1$ for $\sigma > 0$ (so compensation moves toward 0).

For each $n \in \mathbb{N}$, consider the single-event set $E_n = \{e_n\}$ where $\text{apply}(e_n, 0) = -n$. Under the processor model (Definition 3.19), the processor applies e_n to reach state $-n$, then compensates incrementally: $-n \rightarrow -(n-1) \rightarrow \dots \rightarrow -1 \rightarrow 0$. This requires exactly n compensation steps.

For any proposed bound M , choose $n = M + 1$. The compensation episode after this single event contains $n > M$ steps. Therefore no finite M uniformly bounds compensation depth, and UBC fails.

The measure $\Phi(\sigma) = |\sigma|$ strictly decreases under ρ , so WFC holds and compensation terminates for each fixed E_n . The pathology is not non-termination but *unbounded* termination: the compensation depth required to restore validity grows without limit. $\square$

Remark 6.2 (What Fails). The counterexample satisfies WFC (compensation terminates for every fixed E_n) but violates UBC (compensation depth is n , unbounded). In fact every reduction from $(0, \{e_n\})$ to a normal form has exactly n compensation steps, and every measure witnessing WFC for R_∞ is unbounded, not only $|\sigma|$. CC also holds: the only valid state is 0, so $\rho^*(\sigma) = 0$ for every σ , and both sides of CC1 and of CC2 equal 0 for every state and every pair of events, whatever $\text{apply}(e_n, -)$ does away from 0 (take, for instance, $\text{apply}(e_n, \sigma) = \sigma - n$). The pathology is purely one of unbounded depth, which invalidates the uniform step bound and the constant per-event compensation cost in Theorem 7.1. Finite semantic state spaces prevent this by bounding Φ (Lemma 3.8).

6.2 Necessity of CC

Without CC, processors applying the same events in different causality-respecting orders can reach different valid states (Proposition 6.3), and under free delivery this is the only way agreement can fail (Theorem 6.5).

Proposition 6.3 (Disagreement Without CC). *There exists a registry satisfying WFC and UBC, and a finite event set E with two causality-respecting orderings, such that the governance rewrite system reaches different valid normal forms under the two orderings.*

Proof. Let $\Sigma_R = \{A, B, C, D\}$ with valid states $\{A, B\}$ and compensation $\rho_R(C) = A$, $\rho_R(D) = B$. UBC holds with $M = 1$.

Define two events e_1, e_2 with $\text{deps}(e_1) = \text{deps}(e_2) = \emptyset$, so both are enabled from the initial configuration $(A, \{e_1, e_2\})$ and are causally independent under Definition 3.11. Their application is:

	A	B	C	D
e_1	C	C	C	D
e_2	D	C	C	D

Evaluate CC1 at $\sigma = A$. Under the processor model (Definition 3.19), each processor normalizes after each application, so the comparison is between $\rho_R^*(A_{e_2}(\rho_R^*(A_{e_1}(A))))$ and the symmetric order.

Path 1 (e_1 then e_2): $\text{apply}(e_1, A) = C$, $\rho_R^*(C) = A$, $\text{apply}(e_2, A) = D$, $\rho_R^*(D) = B$. Normal form: B .

Path 2 (e_2 then e_1): $\text{apply}(e_2, A) = D$, $\rho_R^*(D) = B$, $\text{apply}(e_1, B) = C$, $\rho_R^*(C) = A$. Normal form: A .

Since $A \neq B$, CC1 is violated. Two processors consuming $\{e_1, e_2\}$ in different causality-respecting orders reach different valid states despite WFC and UBC holding. $\square$

Remark 6.4. The key feature is that ρ_R maps distinct invalid states to *distinct* valid states ($C \mapsto A$, $D \mapsto B$), creating path-dependent repair. Contrast this with the order fulfillment example (Section 4), where compensation preserves approval intent via the held state, ensuring path-independent repair. Theorem 6.5 makes precise in what sense CC captures the boundary between these cases.

Proposition 6.3 is existential: it shows that *some* failure of CC produces disagreement. The converse holds in the following exact form. Say that *every buffered event may fire* (free delivery) when an event is enabled as soon as it is in the buffer, as for events without causal dependencies, and that compensation is *canonical* when ρ_R^* returns a valid state, as Definition 3.15 guarantees under WFC. A state is *reachable* from σ_0 if some reduction from some (σ_0, E) passes through it.

Theorem 6.5 (CC Is Exact). *Let R satisfy WFC, with ρ_R^* as in Definition 3.15, and let every buffered event fire freely. Then every configuration (σ_0, E) with E finite has a unique normal form if and only if CC1 and CC2 hold at every state reachable from σ_0 . Under an arbitrary enabledness predicate (causal or guarded), the system is confluent from a configuration c_0 if and only if, at every configuration reachable from c_0 , the governed successors of every critical pair are joinable; CC1 and CC2 are the special case in which the join is an equality.*

Proof sketch. Sufficiency is Corollary 5.3, with Newman’s Lemma localized to the step-closed set of reachable configurations. For necessity under free delivery, the two sides of CC1 at σ are the normal forms of the runs e_1 then e_2 and e_2 then e_1 from $(\sigma, \{e_1, e_2\})$, and the two sides of CC2 at an invalid σ are the normal forms of apply-then-compensate and compensate-then-apply from $(\sigma, \{e\})$. A reachable σ is reached from σ_0 by delivering some word w first, so a failure of CC at σ yields two distinct normal forms of $(\sigma_0, w \cdot e_1 e_2)$ or $(\sigma_0, w \cdot e)$. The general case is Newman’s Lemma together with the observation that a non-joinable critical pair at a reachable configuration is a pair of distinct normal forms. Both directions are mechanized (`cc_exact_from`, `wfc_cc_exact_from`, `jc_exact`). $\square$

Remark 6.6 (Scope of the Converse). Both qualifiers are needed. If ρ_R^* is only required to be some compensation reduct of its argument (the identity qualifies), CC1 can fail while every configuration has a unique normal form. And under causal enabledness, CC1 can fail for a co-enabled pair at a reachable configuration while every run still reaches the same normal form: events M_1 (write 1) and M_2 (write 2) do not commute, but a reset event that depends on both erases the difference, so from $(0, \{M_1, M_2, M_{\text{reset}}\})$ every normal form is $(0, \emptyset)$ (`rho_star_qualifier`, `masked_cc1`). There the exact condition is the joinability form of Theorem 6.5, not CC1. Together with Theorem 6.1, which satisfies CC and violates UBC, and Proposition 6.3, which satisfies WFC and UBC and violates CC, this shows each condition is needed for its guarantee: UBC for uniform step bounds, CC (exactly, under free delivery) for agreement.

7 Complexity Analysis

Theorem 7.1 (Convergence Complexity). *For a stream prefix of length n under a registry satisfying UBC with bound M and $k = |I_R|$ invariants, the model-level cost of registry-governed*

stream processing is $O(n)$ for event application, validation, and compensation combined. If causality maintenance is implemented with a priority queue, the total cost is $O(n \log n)$, where k and M are treated as constants of system design.

Proof. We analyze the per-event cost, separating model-level costs (which follow from the formalism) from scheduling costs (which depend on implementation).

Model-level per-event costs. For each event e_i ($1 \leq i \leq n$):

- *Event application:* $O(1)$ (state update).
- *Invariant validation:* $O(k) = O(1)$ (evaluate each ψ_j).
- *Compensation:* At most M steps, each $O(1)$. Since M is bounded by UBC, this is $O(1)$.

Registry governance therefore adds $O(1)$ overhead per event beyond what the base stream processor already performs.

Scheduling overhead. Maintaining a set of enabled events and resolving causal dependencies requires implementation-dependent data structures. With a priority queue keyed by timestamp and identifier, each event insertion and dependency-resolution check costs $O(\log i)$, yielding:

Total cost.

$$T(n) = \sum_{i=1}^n (\underbrace{O(1)}_{\text{model}} + \underbrace{O(\log i)}_{\text{scheduling}}) = O(n) + O\left(\sum_{i=1}^n \log i\right) = O(n \log n).$$

The dominant term is causality maintenance. Validation and compensation contribute no asymptotic overhead. Systems with native causal ordering (e.g., single-partition Kafka topics) eliminate the $O(\log i)$ term, reducing the bound to $O(n)$. $\square$

8 Discussion

8.1 Three Regimes of Coordination-Free Convergence

Our result, combined with CRDTs [2] and invariant confluence [5], identifies three distinct structural regimes under which distributed processors converge without coordination:

1. **Operation commutativity** (CRDTs): Operations commute algebraically. Convergence follows from the algebra of operations. No invariant enforcement or compensation is needed.
2. **Invariant confluence** (I-confluence): Operations preserve application invariants under all orderings. Convergence follows because validity is never violated. No compensation is needed.
3. **Normalization confluence** (this paper): Operations may be non-commutative and may violate invariants. Compensation normalizes states to validity. WFC and CC are sufficient for semantic convergence (Theorem 5.5); UBC is orthogonal to convergence and governs scalability (Theorem 7.1). No coordination is needed.

The regimes relate as follows. Operation commutativity with every state valid implies CC (with $\rho_R^* = \text{id}$). Conversely, for a compensation-free system the CRDT conditions are exact: under causal exactly-once delivery it converges from every start state iff its concurrent raw operations commute (the op-based condition), and under unordered at-least-once delivery it converges from σ_0 iff its raw operations are commutative and idempotent on the states reachable from σ_0 , equivalently, for a finite event range with decidable state equality, iff they act by joins in a join-semilattice on those states (the state-based condition) (`crdt_boundary`, parts (a), (b)).

and (b'), from `causal_convergence_exact`, `merge_action_exact` and `cvrtd_on_iff`). The containment is strict on the same transition representation: a one-boolean machine whose raw transitions meet neither condition converges under both deliveries once compensation repairs them (`crdt_boundary`, part (c)). Its governed behavior is itself trivially a CRDT, since every governed step sends every state to the one valid state (`witness_governed_constant`); strictness says that these raw transitions cannot be used without compensation, not that the observable behavior escapes every CRDT. Invariant confluence implies that compensation never fires, but it does not by itself make operations commute: in the sequential-application model of this paper, an invariant-confluent system satisfies CC exactly when its operations commute on the reachable states, and by Theorem 6.5 one whose operations do not commute there diverges under free delivery. So normalization confluence contains the invariant-confluent systems that converge under reordering, not all of them, and it additionally covers systems where operations are non-commutative and invariant-violating, the common case in business applications.

The regimes partition the coordination-free design space by *where* convergence is guaranteed: in the operations (regime 1), in the invariants (regime 2), or in the normalization rewrite system (regime 3).

Remark 8.1 (Convergence as Rewrite Property). The conventional literature frames convergence as a property of operations: operations must commute, or operations must preserve invariants. Normalization confluence reframes convergence as a property of the *rewrite system* that interleaves application and compensation. The operations themselves need no special algebraic structure. This shifts the design burden from operation algebra to compensation design, and places the convergence mechanism at the semantic normalization layer—above the data-structure layer (CRDTs) and the transaction layer (I-confluence).

8.2 Verifying Compensation Commutativity

CC is easy to state but potentially hard to verify: it quantifies over all states and all independent event pairs. Section 9 develops a verification calculus that reduces CC to local per-event and per-pair checks via invariant footprints and decomposable repair. Here we note three common patterns that practitioners can use as a first check before invoking the full calculus:

1. **Compensation to a canonical state.** Suppose there is a default valid state $\sigma_\perp$ such that $\rho_R^*(\sigma) = \sigma_\perp$ for all invalid σ (the “reset to safe default” pattern). This alone does not give CC: on valid states compensation never fires, so two valid-to-valid events that do not commute violate CC1, and an event that treats an invalid state differently from the reset state violates CC2. What the pattern does give is a cheap check. CC2 holds for e if and only if $N(A_e(\sigma)) = N(A_e(\sigma_\perp))$ for every invalid σ (one comparison per invalid state against a fixed reference), where $N = \rho_R^*$ and $A_e = \text{apply}(e, -)$ as in Section 9; and when this holds for e_1 and e_2 , CC1 for the pair at σ holds if and only if $N(A_{e_2}(A_{e_1}(\sigma))) = N(A_{e_1}(A_{e_2}(\sigma)))$. So canonical compensation, the reset check, and commutation up to N together give CC1 and CC2.
2. **Lattice-structured compensation.** Suppose the states form a join-semilattice, each event joins a fixed element ($\text{apply}(e, \sigma) = \sigma \sqcup a_e$), the normalizer N is a closure operator (monotone, extensive, idempotent), and one compensation step stays between a state and its closure ($\sigma \sqsubseteq \rho_R(\sigma) \sqsubseteq N(\sigma)$). Then $N(N(x) \sqcup y) = N(x \sqcup y)$ for all x, y , so both sides of CC1 equal $N(\sigma \sqcup a_{e_1} \sqcup a_{e_2})$, and CC2 holds because $\sigma \sqcup a_e \sqsubseteq \rho_R(\sigma) \sqcup a_e \sqsubseteq N(\sigma) \sqcup a_e$. (This is a short paper-level argument; it is not part of the mechanization.) Monotone repair on an ordered shared domain is also the setting of the companion paper’s [13] cyclic federations, where federated repair reaches a least fixed point (by the Knaster–Tarski theorem) and a further condition decides whether event order matters, with state-based CRDTs as the

compensation-free special case.

3. **Decomposable repair.** If compensation decomposes into commuting per-invariant repairs, CC can be verified via the footprint calculus of Section 9. The order fulfillment example (Section 4) uses this pattern, with one invariant whose repair is the hold.

8.3 Applications

The compensate-and-converge principle applies wherever systems process streams of changes under semantic constraints with bounded compensation. We highlight three domains.

Infrastructure Configuration Drift. Infrastructure-as-code tools apply streams of configuration changes to infrastructure state. Valid configurations are defined by policy. The semantic state space is finite (k boolean policy predicates yield at most 2^k equivalence classes). Remediation operators that reset violated policies to compliant defaults fit the canonical-state pattern: CC2 reduces to one comparison per violating state against the default, and CC1 to commutation of changes up to normalization. When those checks pass, our theorem guarantees convergence to equivalent policy-compliant states regardless of the order in which changes are applied.

Database Schema Migration. Schema migrations are event streams applied to schema state. The set of valid schema configurations is finite. Migration scripts that repair constraint violations constitute compensation operators. When such scripts resolve each constraint independently (decomposable repair), CC reduces to the per-event and per-pair checks of the calculus (Section 9); when those pass, our theorem guarantees order-independent convergence.

Regulatory Compliance. Organizations processing streams of regulatory changes must maintain compliance. Compliance states are finite (each regulation is either satisfied or not). Remediation procedures that restore compliance by addressing each violated regulation independently are decomposable repairs, so CC can be checked with the calculus; when it holds, our theorem guarantees convergence to equivalent compliance states.

8.4 Infinite Domains

Nothing in the convergence result requires Σ_R to be finite. Termination (Lemma 5.1) rests on the well-founded measure $\mu = (|B|, \Phi)$: the buffer B is finite because the received event set is, and Φ maps into a well-founded order—neither depends on $|\Sigma_R| < \infty$. Local confluence (Lemma 5.2) requires CC to *hold*, not to be enumerable. Convergence is therefore domain-independent.

Corollary 8.2 (Domain-Independent Convergence). *Theorem 5.5 and Corollary 5.8 hold for any semantic state space, finite or infinite, provided CC holds and WFC holds with a measure Φ into any well-founded order (not necessarily $\mathbb{N}$).*

This is mechanized at both the rewrite-system and the stream level, with an instance on $\mathbb{Z}$ (Remark 5.4). The necessity example of Section 6 also witnesses it: the registry R_∞ ($\Sigma = \mathbb{Z}$, $\Phi(\sigma) = |\sigma|$, compensation stepping toward 0) satisfies WFC and CC and converges (every event set reaches a unique valid normal form), yet Σ is infinite. What it lacks is UBC: compensation depth grows without bound.

Finiteness therefore underwrites three *secondary* properties, not soundness: (i) UBC and the uniform per-event cost (Theorem 7.1); (ii) verification by exhaustive enumeration; and (iii) the

precomputed normal-form and step tables behind $O(1)$ runtime application. A registry whose logic is captured by k boolean predicates already has a finite semantic quotient— $|\Sigma_R| \leq 2^k$ (Remark 3.3)—so “infinite” here means genuinely *unbounded numeric* semantic state, as in R_∞ .

On such a domain enumeration is unavailable, but CC can still be discharged. The verification calculus (Section 9), with decomposable repair, strong absorption, footprint absorption and commutation up to normalization, establishes CC by *algebraic* conditions that never quantify over an enumerated state space; symbolic decision procedures apply where those do not. When the valid states form a complete lattice and compensation is monotone, the normal form is a fixed point computed by Kleene iteration: finitely under the ascending chain condition, and under *widening* [14] otherwise, at the cost of a sound over-approximation. Without ACC the least fixed point still exists (by the Knaster–Tarski theorem) but finite iteration need not reach it; whenever WFC and CC hold, existence and uniqueness of the normal form are guaranteed by Corollary 8.2. The federated generalization of the monotone case appears in [13].

8.5 Extensions

The single-registry result is the foundation for further extensions. First, *federated streams* where multiple organizations maintain independent registries connected by morphisms; this is developed in the companion paper [13], which gives exact conditions for federated convergence when events fire at federally valid states, and treats multi-source networks (resolution operators) and cyclic networks under monotone repair. Second, *privacy-preserving validation* where organizations prove registry compliance via zero-knowledge proofs, which remains open. Both build on the single-registry convergence result established here.

9 Verification Calculus for CC

Compensation commutativity is a global condition: it quantifies over all states and all independent event pairs. This section develops conditions, exact where possible, that reduce CC to local per-event and per-pair checks. The key tools are an *invariant footprint* mapping and a *decomposable repair* condition.

9.1 Notation

For readability in algebraic manipulations, we write $N(\sigma) := \rho_R^*(\sigma)$ for iterated compensation to validity (Definition 3.15). The normalizer satisfies: (i) $V_R(N(\sigma)) = \top$ for all σ ; (ii) $N(\sigma) = \sigma$ if $V_R(\sigma) = \top$; (iii) $N(N(\sigma)) = N(\sigma)$ (idempotence) (`rho_star_valid`, `rho_star_fix`, `rho_star_idem`). We write $A_e(\sigma) := \text{apply}(e, \sigma)$ when convenient.

In this notation, the CC obligations become:

- (CC1) $N(A_{e_2}(N(A_{e_1}(\sigma)))) = N(A_{e_1}(N(A_{e_2}(\sigma))))$ for independent e_1, e_2 .
- (CC2) $N(A_e(\sigma)) = N(A_e(\rho_R(\sigma)))$ for invalid σ .

We say that CC1 holds for (e_1, e_2) at σ when its equation holds at that state, and that CC2 holds for e when its equation holds at every invalid σ .

9.2 Invariant Vectors and Footprints

Definition 9.1 (Invariant Vector). Define $\mathbf{v} : \Sigma_R \rightarrow \{0, 1\}^k$ by $\mathbf{v}(\sigma)_i = 1 \iff \psi_i(\sigma) = \top$.

Definition 9.2 (Footprint). A *footprint* is a function $F : \mathcal{E} \rightarrow 2^{\{1, \dots, k\}}$ such that for all σ and all $i \notin F(e)$,

$$\mathbf{v}(A_e(\sigma))_i = \mathbf{v}(\sigma)_i.$$

Intuitively, $F(e)$ is the set of invariants whose truth value can change under e . Over-approximations are sound: larger footprints make the calculus more conservative but never incorrect.

9.3 Decomposable Repair

We capture the common case where repair addresses each violated invariant independently.

Definition 9.3 (Per-Invariant Repair). A set of *per-invariant repair operators* $r_1, \dots, r_k : \Sigma_R \rightarrow \Sigma_R$ satisfies:

- (R1) **Targeted fix:** if $\psi_i(\sigma) = \perp$, then $\psi_i(r_i(\sigma)) = \top$.
- (R2) **No-op on satisfied:** if $\psi_i(\sigma) = \top$, then $r_i(\sigma) = \sigma$.
- (R3) **Mutual commutativity:** $r_i(r_j(\sigma)) = r_j(r_i(\sigma))$ for all i, j .

Definition 9.4 (Decomposable Normalizer). The normalizer N is *decomposable* if there exist per-invariant repairs $r_1, \dots, r_k$ as above such that

$$N(\sigma) = r_1(r_2(\dots r_k(\sigma) \dots))$$

for all σ . (The order is irrelevant by mutual commutativity.)

Definition 9.5 (Repair Locality). An event e is *repair-local* with respect to footprint $F(e)$ if for all $i \notin F(e)$,

$$r_i(A_e(\sigma)) = A_e(r_i(\sigma)).$$

That is, e commutes with repairs of invariants it does not affect.

9.4 Strong Absorption and CC2

Theorem 9.6 (Strong Absorption). Assume WFC and let $N = \rho_R^*$. Then CC2 holds (for all events) if and only if, for all events e and states σ ,

$$N(A_e(\sigma)) = N(A_e(N(\sigma))).$$

Proof. ($\Leftarrow$) Let $V_R(\sigma) = \perp$. Because N iterates ρ_R to the unique valid fixpoint reachable from σ , and $\rho_R(\sigma)$ lies on the same compensation chain, we have $N(\sigma) = N(\rho_R(\sigma))$. Applying the assumed equation at σ and at $\rho_R(\sigma)$:

$$N(A_e(\sigma)) = N(A_e(N(\sigma))) = N(A_e(N(\rho_R(\sigma)))) = N(A_e(\rho_R(\sigma))),$$

where the first and third equalities use the assumption and the middle equality uses $N(\sigma) = N(\rho_R(\sigma))$. This is CC2.

($\Rightarrow$) This is the generalization of CC2 noted in Remark 3.18: by induction on the number m of compensation steps from σ to $N(\sigma) = \rho_R^m(\sigma)$, CC2 at each invalid $\rho_R^j(\sigma)$, $j < m$, gives $N(A_e(\rho_R^j(\sigma))) = N(A_e(\rho_R^{j+1}(\sigma)))$, and chaining these equalities gives $N(A_e(\sigma)) = N(A_e(N(\sigma)))$. $\square$

Both directions are mechanized (`strong_absorption_iff_cc2`; the forward direction alone is `base_thm_strong_absorption`). The per-event form also holds: CC2 for a single event e at every invalid state implies strong absorption for e (`calc_cc2_strong_absorption`).

Remark 9.7. Strong absorption states: “normalizing before applying an event does not change the final normalized outcome.” This holds whenever compensation repairs violations in a way determined by the violation pattern rather than by the path to the violation. The order fulfillment example (Section 4) satisfies this: the held state captures the approval intent regardless of whether the order was directly approved with low balance or arrived at the invalid state through a different path.

9.5 Footprints and CC1

We prove two preparatory lemmas before the main theorem.

Lemma 9.8 (Repair-Event Commutation). *If event e is repair-local with respect to footprint $F(e)$ (Definition 9.5), then for any index set J with $J \cap F(e) = \emptyset$ and any state σ :*

$$R_J(A_e(\sigma)) = A_e(R_J(\sigma)),$$

where $R_J = \prod_{i \in J} r_i$.

Proof. By repair locality, $r_i \circ A_e = A_e \circ r_i$ for each $i \in J$ (since $i \notin F(e)$). The result follows by induction on $|J|$: if the claim holds for $R_{J'}$ with $|J'| < |J|$, then for any $j \in J$,

$$R_J(A_e(\sigma)) = r_j(R_{J \setminus \{j\}}(A_e(\sigma))) = r_j(A_e(R_{J \setminus \{j\}}(\sigma))) = A_e(r_j(R_{J \setminus \{j\}}(\sigma))) = A_e(R_J(\sigma)),$$

using the inductive hypothesis and then $r_j \circ A_e = A_e \circ r_j$. $\square$

Lemma 9.9 (Repair Block Idempotence). *For any index set $J \subseteq \{1, \dots, k\}$, the repair block R_J is idempotent on states that are valid on all invariants in J : if $\psi_i(\sigma) = \top$ for all $i \in J$, then $R_J(\sigma) = \sigma$. More generally, $R_J(R_J(\sigma)) = R_J(\sigma)$ for all σ .*

Proof. If $\psi_i(\sigma) = \top$ for all $i \in J$, then each $r_i(\sigma) = \sigma$ by (R2), so $R_J(\sigma) = \sigma$. For general idempotence: $R_J(\sigma)$ satisfies $\psi_i(R_J(\sigma)) = \top$ for all $i \in J$ (each r_i fixes its invariant by (R1) and does not break others by mutual commutativity and (R2)), so $R_J(R_J(\sigma)) = R_J(\sigma)$ by the first claim. $\square$

Theorem 9.10 (Footprint Commutativity). *Let $N = \rho_R^*$ be decomposable via per-invariant repairs $r_1, \dots, r_k$ satisfying (R1) to (R3), let F be a footprint, and let e_1, e_2 be events. Write $F_j = F(e_j)$ and consider the hypotheses*

- (H1) *Disjoint footprints:* $F_1 \cap F_2 = \emptyset$.
- (H2) *Repair locality:* each e_j is repair-local with respect to F_j .
- (H3) *Commutation up to N at σ :* $N(A_{e_2}(A_{e_1}(\sigma))) = N(A_{e_1}(A_{e_2}(\sigma)))$ (raw commutation $A_{e_1} \circ A_{e_2} = A_{e_2} \circ A_{e_1}$ suffices).
- (H4) *Footprint absorption:* $N(A_{e_j}(R_{F_j}(\tau))) = N(A_{e_j}(\tau))$ for all states τ and $j = 1, 2$.

Then:

- (a) **Exact form.** Under (H1) and (H2), for every state σ , CC1 holds at σ if and only if $N(A_{e_2}(A_{e_1}(R_{F_2}(\sigma)))) = N(A_{e_1}(A_{e_2}(R_{F_1}(\sigma))))$.
- (b) **Valid states.** Under (H1) and (H2), for every valid σ , CC1 holds at σ if and only if (H3) holds at σ . In particular, (H1), (H2) and raw commutation give CC1 at every valid state.

- (c) **Every state.** Under (H2), (H3) at σ and (H4), CC1 holds at σ ; (H1) is not needed. Given (H2), footprint absorption (H4) for e_j is equivalent to strong absorption for e_j : $N(A_{e_j}(\tau)) = N(A_{e_j}(N(\tau)))$ for all τ .

Proof. Write J^c for $\{1, \dots, k\} \setminus J$. First, $N \circ R_J = N$ for every index set J : by decomposability and (R3), $N = R_{J^c} \circ R_J$, so $N(R_J(\tau)) = R_{J^c}(R_J(R_J(\tau))) = R_{J^c}(R_J(\tau)) = N(\tau)$ by Lemma 9.9. Second, if e is repair-local with respect to $F(e)$, then for every τ

$$N(A_e(N(\tau))) = N(A_e(R_{F(e)}(\tau))), \quad (1)$$

because $N(\tau) = R_{F(e)^c}(R_{F(e)}(\tau))$, Lemma 9.8 moves $R_{F(e)^c}$ past A_e , and $N \circ R_{F(e)^c} = N$.

(a) By (1) for e_2 , the left side of CC1 is $N(A_{e_2}(N(A_{e_1}(\sigma)))) = N(A_{e_2}(R_{F_2}(A_{e_1}(\sigma))))$, and by (H1) and Lemma 9.8 for e_1 (with $J = F_2$, disjoint from F_1) this is $N(A_{e_2}(A_{e_1}(R_{F_2}(\sigma))))$. Symmetrically, the right side is $N(A_{e_1}(A_{e_2}(R_{F_1}(\sigma))))$.

(b) At a valid σ every $\psi_i(\sigma)$ holds, so $R_{F_1}(\sigma) = R_{F_2}(\sigma) = \sigma$ by Lemma 9.9, and (a) becomes (H3).

(c) By (1), (H4) for e_j says exactly $N(A_{e_j}(N(\tau))) = N(A_{e_j}(\tau))$ for all τ , which is strong absorption (conversely, strong absorption at $R_{F_j}(\tau)$ together with $N(R_{F_j}(\tau)) = N(\tau)$ gives (H4)). Then

$$N(A_{e_2}(N(A_{e_1}(\sigma)))) = N(A_{e_2}(A_{e_1}(\sigma))) = N(A_{e_1}(A_{e_2}(\sigma))) = N(A_{e_1}(N(A_{e_2}(\sigma)))),$$

by strong absorption for e_2 , (H3), and strong absorption for e_1 . $\square$

Corollary 9.11 (CC1 Under CC2). *Assume WFC and let $N = \rho_R^*$. If CC2 holds for e_1 and e_2 , then for every state σ , CC1 holds for the pair at σ if and only if $N(A_{e_2}(A_{e_1}(\sigma))) = N(A_{e_1}(A_{e_2}(\sigma)))$.*

Proof. By Theorem 9.6 (in its per-event form) both events satisfy strong absorption, and the chain of equalities in the proof of Theorem 9.10(c) applies in both directions. $\square$

In a registry satisfying CC2, then, CC1 is exactly commutation up to normalization, and footprints play no role in that equivalence; their role is to make (H4), or CC2 itself, a local check. Every part is mechanized: (a) `calc_footprint_cc1_iff`, (b) `calc_footprint_cc1_valid_iff` and `calc_footprint_cc1_valid`, (c) `calc_footprint_cc1` and `calc_fp_absorb_iff_sa`, Corollary 9.11 `calc_cc1_iff_commute_under_cc2`; with non-vacuity instances `calc_clamp_instance` (two clamped counters with disjoint footprints, CC1 at every state including invalid ones) and `calc_order_cc1`. Form (b) is the one the reference implementation's compositional check uses, since it checks CC1 on valid states. The same reasoning covers a state space split into components: on $\Sigma_1 \times \Sigma_2$ with $N = N_1 \times N_2$ and events acting on one component each, CC1 holds at every valid state, and at any state exactly when each event absorbs its own component's normalizer (`calc_components_cc1_valid`, `calc_components_cc1_iff`).

Remark 9.12 (Each Hypothesis Is Needed). Version 1 of this paper stated Theorem 9.10 with (H1) and (H2) alone as giving CC1 at every state. That is false. With no invariants ($k = 0$) every footprint is empty, repair is the identity, (H1) and (H2) hold vacuously, and the events $x := 1$ and $x := 2$ violate CC1 (`base_thm_footprint_cc1_refuted`). Adding raw commutation does not help: on states $\{0, 1, 2, 3\}$ with $\psi_1 = \psi_2 = (\sigma \geq 2)$, repairs $r_1 = r_2$ sending $0, 1 \mapsto 2$ and fixing 2, 3, and events $a = (0 \mapsto 0, 1 \mapsto 0, 2 \mapsto 2, 3 \mapsto 3)$ (footprint empty, repair-local) and $b = (0 \mapsto 0, 1 \mapsto 0, 2 \mapsto 3, 3 \mapsto 0)$ (footprint $\{1, 2\}$), the raw events commute but at state 0 the two sides of CC1 are 3 and 2 (`base_thm_footprint_cc1_raw_refuted`). The failure is that b does not

absorb the repairs of its own footprint ($N(b(R_{\{1,2\}}(0))) = 3 \neq 2 = N(b(0))$), which is (H4); b also violates CC2 at 0. Dropping any one hypothesis of the corrected forms breaks them, with every other hypothesis holding: the valid-state form without (H3) (`calc_valid_needs_commute`), without (H1) (`calc_valid_needs_disjoint`), or without (H2) (`calc_valid_needs_repair_local`); the every-state form without (H3) (`calc_all_needs_commute`), without (H4) (`calc_all_needs_absorb`, the example above), or without (H2) (`calc_all_needs_repair_local`).

Remark 9.13 (Practical import). A practitioner does not prove CC1 by global state enumeration. Instead: (i) assign each event type an invariant footprint; (ii) verify that each event commutes with repairs outside its footprint (repair locality, Lemma 9.8); (iii) verify, per event, that it absorbs the repairs of its own footprint, (H4), equivalently strong absorption, which CC2 already implies; and (iv) verify, per independent pair, commutation up to normalization, (H3). All four are local obligations, and (iv) is often raw commutation. At valid states, (ii) and disjoint footprints reduce CC1 to (iv) alone. The order fulfillment example (Section 4) does not have disjoint footprints: approval can falsify the single invariant and credit can restore it, so every footprint of either event contains it (`base_rem_practical_import_refuted`). It is certified by part (c) instead: the held-state repair commutes with credit application (`calc_order_credit_repair_commute`), both events absorb their footprint repairs, and the events commute up to normalization, so CC1 holds at every state, and CC2 holds as well (`calc_order_cc1`).

9.6 Product Composition

Real systems comprise multiple semantic modules. The following theorem shows CC verifications compose.

Definition 9.14 (Product Registry). Let $R_1 = (\Sigma_1, I_1, V_1, \rho_1)$ and $R_2 = (\Sigma_2, I_2, V_2, \rho_2)$ with disjoint invariant sets. Define $R_1 \otimes R_2$ by:

$$\Sigma = \Sigma_1 \times \Sigma_2, \quad V(\sigma_1, \sigma_2) = V_1(\sigma_1) \wedge V_2(\sigma_2), \quad \rho(\sigma_1, \sigma_2) = (\rho_1(\sigma_1), \rho_2(\sigma_2)).$$

Events are pairs $e = (e^{(1)}, e^{(2)})$ with application componentwise.

Theorem 9.15 (Product Lifting). *If each R_j satisfies WFC and CC, then $R_1 \otimes R_2$ satisfies WFC and CC; more precisely, CC1 holds for a product pair at (σ_1, σ_2) if and only if it holds for the first components at σ_1 and for the second components at σ_2 . If each normalizer is decomposable, the product normalizer is decomposable with footprints composed by union across components, and repair locality and footprint disjointness lift.*

Proof. WFC: Use measure $\Phi(\sigma_1, \sigma_2) = \Phi_1(\sigma_1) + \Phi_2(\sigma_2)$. Each compensation step strictly decreases at least one component when invalid.

CC2: The product normalizer satisfies $N(\sigma_1, \sigma_2) = (N_1(\sigma_1), N_2(\sigma_2))$ and application is componentwise, so CC2 holds iff it holds in each component.

CC1: Both sides of the CC1 equation decompose into paired equalities in R_1 and R_2 by componentwise application and normalization.

Decomposability: Per-invariant repairs in each component act on disjoint state components and thus commute across components trivially. The normalizer identity $N = N_1 \times N_2$ uses that each ρ_j fixes valid states (Definition 3.1). Mechanized as `base_thm_product` and `base_thm_product_decomposable`, with the instance `calc_product_instance`. $\square$

Remark 9.16. Product composition enables modular system design: verify CC for each semantic module independently, then compose without re-verification. An order processing system with separate inventory, payment, and shipping registries can verify CC per-module and lift to the combined system.

9.7 Partial Decomposability

Full decomposability is a strong sufficient condition. When it does not hold globally, the calculus still provides value:

1. The footprint theorem (Theorem 9.10) is a statement about one event pair: it applies to any pair satisfying its hypotheses, regardless of whether *all* pairs do, and Corollary 9.11 needs no decomposability at all.
2. When footprints overlap for some event pair, the calculus identifies precisely which pairs require further analysis. A practitioner can then check the corrected canonical-state pattern for the overlapping invariants (Section 8.2, pattern 1) or add coordination only for the interacting event types.
3. For event types that share footprints but whose normalized outcomes agree by domain-specific reasoning (e.g., both repairs clamp to the same bound), CC1 can be verified by case analysis restricted to the overlapping states. The footprint calculus reduces the case space.

10 Conclusion

We have identified *normalization confluence* as a third structural regime for coordination-free convergence in distributed systems, alongside operation commutativity (CRDTs) and invariant confluence (I-confluence). The regime is characterized by two conditions: well-founded compensation (WFC), which ensures termination, and compensation commutativity (CC), which ensures confluence. Under both, the governance rewrite system has unique normal forms by Newman’s Lemma, and all processors consuming the same events converge to the same valid state. A uniform bound on compensation depth (UBC) additionally guarantees constant per-event overhead. Without UBC, compensation depth is unbounded. CC is exact: when every buffered event may fire and compensation repairs to validity, every event set has a unique normal form if and only if CC holds on the reachable states, and under causal or guarded enabledness a joinability form of CC is the exact condition. Convergence does not depend on the state space being finite; finiteness serves only uniform cost and checking by enumeration.

To make CC practically verifiable, we developed a verification calculus that reduces the global CC obligation to local checks. Invariant footprints identify which invariants each event can affect; decomposable repair ensures per-invariant fixes commute; CC2 is equivalent to strong absorption; CC1 at every state follows from repair locality, per-event footprint absorption and commutation up to normalization, and at valid states disjoint footprints and repair locality reduce it to commutation up to normalization alone; and product composition enables modular verification across semantic modules. Each hypothesis is shown to be needed. Together, these convert “compensate-and-converge” from a convergence theorem into a verifiable design pattern with tractable proof obligations. The convergence theory and the calculus are mechanized in Coq/Rocq without axioms, including the paper’s examples and the counterexamples that delimit each theorem.

Compensation commutativity is strictly weaker than the operation commutativity required by CRDTs: operations need not commute, only their compensated results. The gap between these two conditions is precisely the design space that normalization confluence occupies—systems where operations are non-commutative, invariant-violating, but convergent under repair.

The three regimes partition the coordination-free design space by where convergence is guaranteed: in the operations, in the invariants, or in the normalization rewrite system. This last regime formalizes a pattern that practitioners use implicitly—post-application repair as a con-

vergence mechanism—and provides both the structural conditions under which it is sound and a calculus for verifying those conditions.

Acknowledgments

I thank my wife, Yuval, for her patience and support throughout this research.

A Errata and corrections

This appendix lists every statement of version 1 that version 2 refutes, corrects or sharpens. For each, it gives the old statement (short), the counterexample where there is one, and the corrected result, with the names of the Coq theorems (all in the axiom-free gate of `coq/verify.sh`). The body of the paper states only the corrected results.

- E1. Theorem 9.10 (Footprint Commutativity).** *Old:* under WFC and decomposable repair, disjoint footprints (H1) and repair locality (H2) give CC1 for an independent pair. *Counterexamples:* with $k = 0$ the events $x := 1$ and $x := 2$ satisfy (H1) and (H2) vacuously and violate CC1 (`base_thm_footprint_cc1_refuted`); the four-state example of Remark 9.12 satisfies (H1), (H2) and raw commutation and violates CC1 at state 0 (`base_thm_footprint_cc1_raw_refuted`). The old proof’s final step moved a repair block past an event whose footprint it intersects, which repair locality does not allow. *Corrected:* Theorem 9.10: (a) under (H1), (H2), CC1 at σ iff the events commute up to N after the other event’s footprint repairs (`calc_footprint_cc1_iff`); (b) at valid states, CC1 iff commutation up to N (`calc_footprint_cc1_valid_iff`, `calc_footprint_cc1_valid`); (c) at every state, repair locality, commutation up to N and per-event footprint absorption give CC1, without (H1) (`calc_footprint_cc1`, `calc_fp_absorb_iff_sa`); and Corollary 9.11: under CC2, CC1 iff commutation up to N (`calc_cc1_iff_commute_under_cc2`). Each hypothesis is needed (`calc_valid_needs_commute`, `calc_valid_needs_disjoint`, `calc_valid_needs_repair_local`, `calc_all_needs_commute`, `calc_all_needs_absorb`, `calc_all_needs_repair_local`).
- E2. Remark “Practical import”.** *Old:* the order-fulfillment events have disjoint footprints, so the footprint workflow certifies the example. *Counterexample:* the single invariant lies in every footprint of both events (`base_rem_practical_import_refuted`). *Corrected:* Remark 9.13: the workflow has a fourth step (per-event footprint absorption), and the example is certified by Theorem 9.10(c) (`calc_order_credit_repair_commute`, `calc_order_cc1`).
- E3. Section 8.2, pattern 1 (compensation to a canonical state).** *Old:* if $\rho_R^*(\sigma) = \sigma_\perp$ for every invalid σ , CC1 and CC2 hold trivially. *Counterexamples:* $x := 1$ and $x := 2$ on valid states violate CC1 (`base_pattern1_cc1_refuted`); an event that sends an invalid state and the reset state to different places violates CC2 (`base_pattern1_cc2_refuted`). *Corrected:* CC2 for e iff $N(A_e(\sigma)) = N(A_e(\sigma_\perp))$ for every invalid σ (`calc_canonical_cc2_iff`); given that for both events, CC1 iff commutation up to N (`calc_canonical_pattern`, `instance calc_canonical_instance`).
- E4. Section 8.2, pattern 2 (lattice-structured compensation).** *Old:* compensation that “computes the join with the nearest valid state” satisfies CC by commutativity of the join. This was imprecise (“nearest valid state” is undefined). *Corrected:* stated precisely

(events join fixed elements, N a closure operator, $\sigma \sqsubseteq \rho_R(\sigma) \sqsubseteq N(\sigma)$) with a short paper proof; not mechanized. The federated remark in the same item no longer claims event-order convergence on monotone cycles, which the companion paper’s revision shows needs a further condition.

- E5. Section 4, “Why Compensation Design Matters”.** *Old:* under the naive revert $\rho(\text{approved}, b) = (\text{pending}, b)$, CC1 fails at $(\text{pending}, 0)$. *Counterexample:* at $(\text{pending}, 0)$ both orders give $(\text{pending}, 1)$ (`naive_paper_witness_fails`). *Corrected:* CC1 fails at $(\text{pending}, 1)$, with $(\text{pending}, 2) \neq (\text{approved}, 2)$ (`naive_cc1_fails`); the paper’s stream diverges (`naive_stream_diverges`); the naive revert also violates CC2 at $(\text{approved}, 1)$ (`naive_cc2_fails`). The closing paragraph no longer cites “state-determined compensation” as a sufficient condition (canonical compensation is a special case and is not sufficient, E3); it names the absorption and commutation facts that Theorem 9.10 uses. The exhaustive check covers all 12 states (`of_cc1`, `of_cc2`, `of_processors`).
- E6. Remark “What Fails” (after Theorem 6.1).** *Old:* R_∞ satisfies CC because “there is only one event per set, so order independence is vacuous”. The conclusion is true, the reason is not (CC1 is a property of the registry over all events, and $\text{apply}(e_n, -)$ was defined only at 0). *Corrected:* ρ^* is constantly 0, so CC1 and CC2 hold for every total extension of $\text{apply}(e_n, -)$ (`ri_cc_any_extension`, `ri_what_fails`); stronger forms of the UBC failure: `ri_depth_exact`, `ri_no_ubic`. Theorem 6.1 itself is unchanged (`thm_necessity`).
- E7. Theorem 5.5(c) (Agreement).** *Old:* if $E_{P_1}(t) = E_{P_2}(t)$ then $\Psi_{P_1}(t) = \Psi_{P_2}(t)$. *Counterexample:* one processor has applied the single received event and the other has not yet (`base_thm_convergence_c_counterexample`). *Corrected:* processors that have *finished processing* equal received sets agree, at the same or different times (`base_thm_convergence_c`, `stream_agreement`); parts (a) and (b) are stated for finished processors (`base_thm_convergence`, `stream_convergence`). Definition 3.19 now defines “finished” and “fair”.
- E8. Closing sentence of Theorem 5.5 and Remark 5.9.** *Old:* with eventual delivery, disagreements are transient: any event causing processors to differ is eventually received by both, restoring agreement. *Counterexample:* on an infinite stream, a processor one event behind another is finished at every time and disagrees at every time (`base_thm_convergence_transient_counterexample`). *Corrected:* the sentence is removed; Remark 5.6 and Remark 5.9 state what holds: (c) as corrected, and fixed-point agreement for finite streams.
- E9. Corollary 5.8.** *Old:* for a finite stream the processors eventually agree forever. The proof assumed both processors had finished processing at T_N . *Corrected:* the same conclusion for fair processors (`base_cor_quiescent`, `base_cor_quiescent_nat`).
- E10. “CC is necessary” (abstract, contributions, Section 6, the remark after Proposition 6.3, conclusion).** *Old:* CC is necessary for agreement, and each condition is independently necessary for its guarantee. Proposition 6.3 is existential and remains true (`prop_cc_necessary`), but CC as stated is not necessary in the paper’s causal setting. *Counterexamples:* under causal enabledness CC1 can fail at a reachable configuration while normal forms are unique (`masked_cc1`); with non-canonical ρ^* the same (`rho_star_qualifier`). *Corrected:* Theorem 6.5: under free delivery and canonical repair, unique normal forms from σ_0 iff CC1 and CC2 on the states reachable from σ_0 (`cc_exact_from`, `wfc_cc_exact_from`); under any enabledness, confluence iff joinability at reachable configurations (`jc_exact`); Remark 6.6.

- E11. Theorem 9.6.** *Old:* strong absorption implies CC2. *Sharpened:* under WFC the two are equivalent (`strong_absorption_iff_cc2`; forward direction `base_thm_strong_absorption`; per event `calc_cc2_strong_absorption`).
- E12. Three-regime hierarchy (Section 1, Section 8.1, Related Work).** *Old:* the regimes form a containment hierarchy of progressively weaker requirements, and normalization confluence subsumes invariant confluence. This was imprecise: an invariant-confluent system needs no compensation but its operations need not commute, so it can violate CC1. Related Work also called operations that individually violate invariants something “CRDTs cannot accommodate”. The difference is the convergence mechanism (repair versus operation algebra), not what the behavior can be: a governed machine’s behavior can itself be a CRDT (`witness_governed_constant`). *Corrected:* normalization confluence contains the invariant-confluent systems that converge under reordering. On the CRDT side, a compensation-free system converges under causal exactly-once delivery iff it meets the op-based condition, and under unordered at-least-once delivery iff it meets the state-based condition on the reachable states (`crdt_boundary`, parts (a), (b) and (b’); `causal_convergence_exact`, `merge_action_exact`, `cvrtdt_on_iff`); the embedding is `cmrtdt_governed_SEC.compensation_free_exact` only identifies governed steps with raw operations once compensation is the identity, a structural step and not a convergence theorem. The containment is strict on the same transition representation (`crdt_boundary`, part (c); `witness_converges`, `witness_not_cmrdt`), and the witness’s governed behavior is itself a CRDT (`witness_governed_constant`).
- E13. Sharpened or newly mechanized, no statement refuted.** Definition 3.15: ρ_R^* is constructed from WFC, needs at most $\Phi(\sigma)$ steps, and is independent of the measure (`base_def_rhostar`, `base_def_rhostar_least_unique`, `rho_star_canonical`). Lemma 5.1: the bound is attained (`base_lem_termination_bound`, `lv_termination_bound_tight`). Corollary 8.2 (domain independence) is mechanized at the rewrite-system and stream levels (`governance_wf_confluent`, `causal_governance_wf_confluent`, `base_cor_infinite`, `base_cor_infinite_quiescent`, `zw_confluent`) and now also covers Corollary 5.8; its paragraph on widening no longer claims uniqueness of the normal form without WFC. Theorem 9.15: CC1 lifts as an equivalence with the componentwise conditions (`base_thm_product`, `base_thm_product_decomposable`). The Applications paragraphs no longer assert CC for reset and per-constraint remediation; they point to the corrected checks. The remark “Machine-checked” (Remark 5.4) lists the Coq theorem names; version 1’s phrase “footprint disjointness implies order-independent commutation” referred to state-variable footprints (`disjoint_events_commute`), not to Theorem 9.10.

References

- [1] M. H. A. Newman, “On theories with a combinatorial definition of ‘equivalence’,” *Annals of Mathematics*, vol. 43, no. 2, pp. 223–243, 1942.
- [2] M. Shapiro, N. Preguiça, C. Baquero, and M. Zawirski, “A comprehensive study of convergent and commutative replicated data types,” *INRIA Research Report RR-7506*, 2011.
- [3] J. M. Hellerstein, “The declarative imperative: experiences and conjectures in distributed logic,” *SIGMOD Record*, vol. 39, no. 1, pp. 5–19, 2010.
- [4] T. J. Ameloot, F. Neven, and J. Van den Bussche, “Relational transducers for declarative networking,” *Journal of the ACM*, vol. 60, no. 2, pp. 1–38, 2013.

- [5] P. Bailis, A. Fekete, M. J. Franklin, A. Ghodsi, J. M. Hellerstein, and I. Stoica, “Coordination avoidance in database systems,” *Proc. VLDB Endowment*, vol. 8, no. 3, pp. 185–196, 2014.
- [6] F. Baader and T. Nipkow, *Term Rewriting and All That*, Cambridge University Press, 1998.
- [7] W. Vogels, “Eventually consistent,” *Communications of the ACM*, vol. 52, no. 1, pp. 40–44, 2009.
- [8] H. Garcia-Molina and K. Salem, “Sagas,” *Proc. ACM SIGMOD International Conference on Management of Data*, pp. 249–259, 1987.
- [9] E. A. Lee and D. G. Messerschmitt, “Synchronous data flow,” *Proceedings of the IEEE*, vol. 75, no. 9, pp. 1235–1245, 1987.
- [10] G. Kahn, “The semantics of a simple language for parallel programming,” *Information Processing*, pp. 471–475, 1974.
- [11] T. Akidau, R. Bradshaw, C. Chambers, et al., “The Dataflow Model: A practical approach to balancing correctness, latency, and cost in massive-scale, unbounded, out-of-order data processing,” *Proc. VLDB Endowment*, vol. 8, no. 12, pp. 1792–1803, 2015.
- [12] L. Lamport, “Time, clocks, and the ordering of events in a distributed system,” *Communications of the ACM*, vol. 21, no. 7, pp. 558–565, 1978.
- [13] Dayna Blackwell, “Normalization Confluence in Federated Registry Networks,” Zenodo, 2026. DOI: 10.5281/zenodo.18677400.
- [14] P. Cousot and R. Cousot, “Abstract interpretation: a unified lattice model for static analysis of programs by construction or approximation of fixpoints,” *Proc. 4th ACM SIGACT-SIGPLAN Symp. on Principles of Programming Languages (POPL)*, pp. 238–252, 1977.
- [15] V. Balegas, S. Duarte, C. Ferreira, R. Rodrigues, N. Preguiça, M. Najafzadeh, and M. Shapiro, “Putting consistency back into eventual consistency,” *Proc. 10th European Conf. on Computer Systems (EuroSys)*, 2015.
- [16] V. Balegas, S. Duarte, C. Ferreira, R. Rodrigues, and N. Preguiça, “IPA: Invariant-preserving applications for weakly consistent replicated databases,” *Proc. VLDB Endowment*, vol. 12, no. 4, 2018.
- [17] J. Meseguer, “Conditional rewriting logic as a unified model of concurrency,” *Theoretical Computer Science*, vol. 96, no. 1, pp. 73–155, 1992.
- [18] F. Durán and J. Meseguer, “On the Church-Rosser and coherence properties of conditional order-sorted rewrite theories,” *Journal of Logic and Algebraic Programming*, vol. 81, no. 7–8, pp. 816–850, 2012.
- [19] E. W. Dijkstra, “Self-stabilizing systems in spite of distributed control,” *Communications of the ACM*, vol. 17, no. 11, pp. 643–644, 1974.
- [20] V. B. F. Gomes, M. Kleppmann, D. P. Mulligan, and A. R. Beresford, “Verifying strong eventual consistency in distributed systems,” *Proc. ACM on Programming Languages*, vol. 1, no. OOPSLA, art. 109, 2017.